

CSE 132A Review

Kevin Jacob

Fall 2025

Table of Contents

1. Relational Algebra
2. SQL
3. Conceptual Modeling
4. Relational Database Design Dependencies
5. Relational Database Design Normalization
6. Recursive Queries
7. Logical Query Optimization
8. Physical Query Optimization
9. Indexing
10. Transactions
11. Concurrency Control and Database Recovery

1 Relational Algebra

1.1 Algebra Concepts

- **Algebra:** An algebra is a mathematical structure that consists of
 - A set (A): The domain or universe of elements
 - A Signature (F): A set of operations defined on the set A
 - Axioms (optional): Properties that the operations must satisfy

- An algebra is minimal if it has no proper subalgebras

Check if algebra is minimal:

Let $\mathbf{A} = (A, F)$ be an algebra, where A is a nonempty set (the *universe*) and F is a set of operations on A (each $f \in F$ has arity n_f and is a map $f : A^{n_f} \rightarrow A$).

A subset $B \subseteq A$ forms a *subalgebra* $\mathbf{B} = (B, F_B)$ of $\mathbf{A}$ if and only if

$$\forall f \in F, \forall b_1, \dots, b_{n_f} \in B, \quad f(b_1, \dots, b_{n_f}) \in B.$$

That is, B is closed under all operations in F .

The algebra $\mathbf{A}$ is called *minimal* if and only if:

$$|A| > 1 \quad \text{and} \quad \forall B \subseteq A, \quad (B \text{ is a subalgebra of } \mathbf{A}) \Rightarrow (|B| = 1 \text{ or } B = A).$$

In words, $\mathbf{A}$ is nontrivial and has no proper nontrivial subalgebras.

To check whether $\mathbf{A}$ is minimal:

1. List all subsets $B \subseteq A$ with $1 < |B| < |A|$.
2. For each B , verify closure:

$$\forall f \in F, \forall b_1, \dots, b_{n_f} \in B, \quad f(b_1, \dots, b_{n_f}) \in B.$$

3. If no such B satisfies closure, then $\mathbf{A}$ is minimal.

Example. For $\mathbf{A} = (\mathbb{Z}_p, +_p)$ where p is prime, the only subgroups of $(\mathbb{Z}_p, +_p)$ are $\{0\}$ and $\mathbb{Z}_p$. Therefore, $\mathbf{A}$ is minimal.

- **Tuple:** A tuple is a finite ordered collection of elements, where each element can be of a different type
- **Many-sorted Algebra:** A many-sorted algebra generalizes the concept of a single-sorted algebra by allowing multiple *carrier sets* (sorts), with operations that can involve elements from different sorts

1.2 Relational Algebra Overview

- Relational ALgebra is a many-sorted algebra
- Sorts (S)
 - Domains (D): Sets of atomic values for attributes
 $D_A = \{1, 2, 3\}, D_B = \{\text{"Apple"}, \text{"Banana"}\}$
 - Attributes (A): Names of attributes, associated with specific domains
 $A_{\text{Attributes}} = \{(A, D_A), (B, D_B)\}$
 - Tuples (T): Ordered collections of values, one for each attribute, constrained by attribute-domain mappings
 $A_{\text{Tuples}} = \{t : A_{\text{Attributes}} \rightarrow A_{\text{Values}} \mid t(A) \in D_A, \forall A \in A_{\text{Attributes}}\}$
 - Relations (R): Sets of tuples constrained by a schema (attribute names and domains)
 $A_{\text{Relations}} = \{R \subseteq A_{\text{Tuples}} \mid \text{Schema}(R) = \text{Attributes of } R\}$

1.3 Core Relational Algebra

- Union, intersection, and difference
 - Usual set operations, but both operands must have the same relation schema
- Selection: $R1 := \sigma_C(R2)$
 - C is a condition that refers to attributes of $R2$
 - $R1$ is all those tuples of $R2$ that satisfy C
- Projection $R1 := \pi_L(R2)$
 - L is a list of attributes from the schema of $R2$
 - $R1$ is constructed by looking at each tuple of $R2$ and extracting the attributes on list L in the order specified
 - Eliminate duplicate tuples, if any
- Extended Projection
 - Using the π_L operator, the list L can contain arbitrary expressions involving attributes ($A + B \rightarrow C$)
 - Can also concatenate attributes of duplicate occurrences of the same attributes
- Cartesian Product $R3 := R1 \times R2$
 - Pairs each tuple $t1$ of $R1$ with each tuple $t2$ of $R2$
 - Concatenation of $t1$ and $t2$ is a tuple of $R3$
 - Schema of $R3$ is the attributes of $R1$ and then $R2$ in order
 - Note: be careful with attribute A of same name and use $R1.A$ and $R2.A$
- Theta Join $R3 := R1 \bowtie_C R2$
 - Takes the product of $R1 \times R2$ then applies σ_C to the result
 - C can be any boolean valued condition
- Natural Join $R3 := R1 \bowtie R2$
 - Connects to relations by equating attributes of the same name
 - Projects out one copy of each pair of equated attributes
- Renaming $R1 : \rho_{R1(A1, \dots, An)}(R2)$
 - Gives a new schema to the relation
 - Makes $R1$ a relation with attributes $A1, \dots, An$ and the same tuples as $R2$
- Relational Division $R \div S$
 - Given R is a relation over attributes $X \cup Y$ and S is a relation over attributes Y
 - $R \div S = \{x \in \pi_X(R) \mid \forall_y \in S, (x, y) \in R\}$
 - Finds all x values in R that are associated with every y in S

2 SQL

2.1 Creating a Table

When creating a table, we can specify constraints on attributes. It is also important to note that there are differences in the implementation of the data type so it is important to consider what data type is more efficient (ex. more efficient to use varchar instead of text if we have know max size)

```
1 create table "Airbnb_listings"
2 (
3     id integer,
4     listing_url text,
5     name varchar(256) not null,
6     summary text,
7     state char(2),
8     amenities text[],
9     requires_license boolean
10    reviews_per_month numeric
11 )
```

Note: text[] is a non-atomic collection datatype

We can add a uniqueness constraint as follows

```
1 create unique index airbnb_listings_listings_url_index
2 in "Airbnb_listings" (listing_url);
```

```
1 ALTER TABLE "Airbnb_listings"
2 ADD CONSTRAINT Listing_pk PRIMARY KEY (listing_url);
```

Note: Can also specify a primary key when creating table in line PRIMARY KEY (A) where A is comma separated attributes that make up key

2.2 Standard Data Types

Type	Use
Integer	int
Numeric	floating point numbers
Varchar	strings with known max length
Text	strings with large maximum length
Date, Time, Datetime	timestamps
Point, Line, Polygon, Circle	shapes
Binary	0,1
Arrays	multidimensional list of any of the above types

Each data type allows a specific set of operations

2.3 Basic SQL Construct

SELECT <attribute-list> ← Attributes we want returned from query

FROM <table-list> ← Tables we want to use

WHERE <predicates> ← Predicates to be satisfied from every tuple result

- The result of an SQL query is a table
- Query: a mapping from a set of input relations and a set of predicates to an output relation
- The order of tuples in the result relation is arbitrary

2.3.1 Projection Query

Ex: Find the ID, Name, space, description, room type, and price of Airbnb places (Can also rename columns using AS)

```
1 SELECT id, name, space, room_type, price
2 FROM "Airbnb_listings";
```

2.3.2 Select Project Query

Find the ID, Name, space description, and price of Airbnb places only for private rooms (don't show null values for space)

```
1 SELECT id, name, space, price
2 FROM "Airbnb_listings"
3 WHERE room_type = 'Private room' AND space IS NOT NULL;
```

If we want to add an additional predicate that the place should be in DC or MD and also return the city/state (note parentheses are important for association)

```
1 SELECT id, name, space, city, state, price
2 FROM "Airbnb_listings"
3 WHERE room_type = 'Private room' AND space IS NOT NULL AND (state = 'MD' OR state = 'DC');
```

2.3.3 Fixed Number of Records

We can return a fixed number of results by using limit

```
1 SELECT id, name, space, city, price
2 FROM "Airbnb_listings"
3 WHERE room_type = 'Private Room'
4 LIMIT 10;
```

2.4 Sets, Multisets, and Deduplication

- **Set:** a collection of homogeneous objects with no duplicates
- **Multiset (Bag):** a collection of homogeneous objects with multiple instances of the same object (ALL used with set operators to preserve duplicates)
- **Deduplication:** a function to convert a multiset to a set with the same elements (DISTINCT keyword)

Get the room types will return rows with the same value for room type, so we can add DISTINCT to get unique values

```
1 SELECT DISTINCT room_type
2 FROM "Airbnb_listings";
```

2.5 String Operations

2.5.1 Functions

The query below filters to listings where room_type is a private room and the price is less than 100. regex_replace replaces commands with empty character, ltrim is used to remove leading \$, and ::numeric is the type cast operator

```
1 SELECT id, name, space, city, price
2 FROM "Airbnb_listings"
3 WHERE room_type = 'Private Room' AND space IS NOT NULL AND ltrim(regex_replace(price, '$', ''))
   , '$')::numeric < 100;
```

Below are some additional examples of string functions

Function	Return Type	Description	Example	Result
<code>string string</code> ←	text	String concatenation	<code>'Post' 'greSQL'</code>	PostgreSQL
<code>string non-string or non-string string</code>	text	String concatenation with one non-string input	<code>'Value: ' 42</code>	Value: 42
<code>bit_length(string)</code>	int	Number of bits in string	<code>bit_length('jose')</code>	32
<code>char_length(string) or character_length(string)</code>	int	Number of characters in string	<code>char_length('jose')</code>	4
<code>lower(string)</code> ←	text	Convert string to lower case	<code>lower('TOM')</code>	tom
<code>octet_length(string)</code>	int	Number of bytes in string	<code>octet_length('jose')</code>	4
<code>overlay(string placing string from int [for int])</code>	text	Replace substring	<code>overlay('Txxxxas' placing 'hom' from 2 for 4)</code>	Thomas
<code>position(substring in string)</code>	int	Location of specified substring	<code>position('om' in 'Thomas')</code>	3
<code>substring(string [from int] [for int])</code>	text	Extract substring	<code>substring('Thomas' from 2 for 3)</code>	hom
<code>substring(string from pattern)</code> ↑	text	Extract substring matching POSIX regular expression. See Section 9.7 for more information on pattern matching.	<code>substring('Thomas' from '...\$')</code>	mas
<code>substring(string from pattern for escape)</code>	text	Extract substring matching SQL regular expression. See Section 9.7 for more information on pattern matching.	<code>substring('Thomas' from '%#o_a#_' for '#')</code>	oma
<code>trim([leading trailing both] [characters] from string)</code> ←	text	Remove the longest string containing only characters from <i>characters</i> (a space by default) from the start, end, or both ends (both is the default) of <i>string</i> .	<code>trim(both 'xyz' from 'yxTomxx')</code>	Tom

2.5.2 Comparison

- In addition to using = and != for string comparison, we can also use the LIKE comparison operator for partial matches
- % is used to match an arbitrary number of characters
- _ is used to match a single arbitrary character
- If the characters appear in the string values we want to search for, we need to escape using \
- ilike is the case insensitive version of LIKE

String Comparison Examples

LIKE Operator	Description
WHERE EName LIKE 'a%'	Finds any values that start with "a"
WHERE EName LIKE '%a'	Finds any values that end with "a"
WHERE EName LIKE '%or%'	Finds any values that have "or" in any position
WHERE EName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE EName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE EName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

2.6 Keys

- **Superkey:** A superkey of a relation schema $R = \{A_1, A_2, \dots, A_N\}$ is a set of attributes S subset of R with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[s] = t_2[s]$
- **Key:** A key K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey anymore (a key is a minimal superkey)
- **Primary Key:** One of the candidate keys selected by the database
 - Uniqueness: no two tuples in R can have the same value for the primary key
 - Minimality: no attribute can be removed from the key without losing uniqueness
 - Non-nullability: every tuple must have non-null values for the primary key attributes(s)

– Stability: primary key values should not change frequently since they serve as primary identifiers

- **Prime Attribute:** attribute that is a member of some candidate key
- **Nonprime attribute:** not a prime attribute

2.7 Functions

Examples of built in functions

- Numeric: *sqr()*, *sqrt()*, *abs()*, *round()*, ...
- String: *trim()*, *lower()*, *upper()*, ...
- Casting: *::numeric* ...
- Date: data '2001-09-28' + interval '1 hour'

User Defined Functions

Below is an example of defining a function for the price parsing shown above

```
1 CREATE FUNCTION conv_price(text) RETURNS numeric
2 AS 'select ltrim(regexp_replace($1,"",""),"$")::numeric;'
3 LANGUAGE SQL
4 IMMUTABLE
5 RETURNS NULL ON NULL INPUT;
6
7 SELECT DISTINCT conv_price(weekly_price) FROM "Airbnb_listings";
```

2.8 Aggregate Functions

Functions that can be formed on collections of values such as count, sum, and avg

```
1 SELECT count(*), round(avg(conv_price(price)),3) as avgPrice, round(stddev(conv_price(price))
2 ,3) as stdevPrice, max(conv_price(price)) as maxPrice, min(conv_price(price)) as
   minPrice
3 FROM "Airbnb_listings"
4 WHERE price IS NOT NULL AND room_type LIKE '%home' AND state = 'DC';
```

Example of finding median values of groups (mentioned below)

```
1 SELECT fund_family, percentile_cont(0.5) WITHIN GROUP (ORDER BY fund_price_earning_ratio) as
   group_median
2 FROM mutualfunds
3 GROUP BY fund_family
4 ORDER BY group_median desc
5 LIMIT 10;
```

2.9 Groups

2.9.1 Group By/Order By

The Group By operation extracts unique values of the grouping attributes and creates lists of the grouped attributes. Aggregate functions can then be applied to each group. (The select clause can only have grouping attributes and the aggregate functions)

```
1 SELECT city, property_type, room_type, count(*), round(avg(conv_price(price)),3) as avgPrice
   , round(stddev(conv_price(price)),3) as stdevPrice, max(conv_price(price)) as maxPrice,
   min(conv_price(price)) as minPrice
2 FROM "Airbnb_listings"
3 WHERE price IS NOT NULL
4 GROUP BY city, property_type, room_type
5 ORDER BY city asc, property_type desc;
```

2.9.2 Having clause

Used to filter groups (can only be applied to aggregate functions in the select clause since applied as part of the SELECT clause). The WHERE clause is different because it applies predicates on data values and can be applied to any attributes regardless of appearance in SELECT clause (applied before SELECT).

```
1 SELECT city, property_type, room_type, count(*), round(avg(conv_price(price)),3) as avgPrice
   , round(stddev(conv_price(price)),3) as stdevPrice, max(conv_price(price)) as maxPrice,
   min(conv_price(price)) as minPrice
2 FROM "Airbnb_listings"
3 WHERE price IS NOT NULL AND NOT(summary ilike '%museum%') and property_type != 'Bed &
   Breakfast'
4 GROUP BY city, property_type, room_type
5 HAVING count(*) > 5
6 ORDER BY city asc, property_type desc;
```

Note: example above used NOT for negation and we could do NOT(summary ilike '%museum%' OR property_type = 'Bed & Breakfast')

2.10 Set Operations

2.10.1 Union

```
1 (SELECT 'price' as PriceType, count(*), round(avg(conv_price(price)),3) as avgPrice, round(
   stddev(conv_price(price)),3) as stdevPrice, max(conv_price(price)) as maxPrice, min(
   conv_price(price)) as minPrice
2 FROM "Airbnb_listings"
3 WHERE price IS NOT NULL)
4 UNION
5 (SELECT 'weekly price' as PriceType, count(*), round(avg(conv_price(weekly_price)),3) as
   avgPrice, round(stddev(conv_price(weekly_price)),3) as stdevPrice, max(conv_price(
   weekly_price)) as maxPrice, min(conv_price(weekly_price)) as minPrice
6 FROM "Airbnb_listings"
7 WHERE weekly_price IS NOT NULL)
8 UNION
9 (SELECT 'monthly price' as PriceType, count(*), round(avg(conv_price(monthly_price)),3) as
   avgPrice, round(stddev(conv_price(monthly_price)),3) as stdevPrice, max(conv_price(
   monthly_price)) as maxPrice, min(conv_price(monthly_price)) as minPrice
10 FROM "Airbnb_listings"
11 WHERE monthly_price IS NOT NULL);
```

2.10.2 Difference

first set - second set

```
1 (SELECT DISTINCT city, property_type, room_type
2 FROM "Airbnb_listings"
3 WHERE price IS NOT NULL
4 ORDER BY city asc, property_type desc)
5 EXCEPT
6 (SELECT DISTINCT city, property_type, room_type
7 FROM "Airbanb_listings"
8 WHERE price IS NOT NULL AND NOT (summary ilike '%musesums%') AND property_type != 'Bed &
   Breakfast'
9 ORDER BY city asc, proeprty_type desc);
```

2.10.3 Intersection

```
1 (SELECT DISTINCT city, property_type, room_type
2 FROM "Airbnb_listings"
3 WHERE price IS NOT NULL
4 ORDER BY city asc, property_type desc)
5 INTERSECT
6 (SELECT DISTINCT city, property_type, room_type
7 FROM "Airbnb_listings"
8 WHERE price IS NOT NULL AND NOT (summary ilike '%musesums%') AND property_type != 'Bed &
9   Breakfast'
10 ORDER BY city asc, proeprty_type desc);
```

2.11 Random Sampling

```
1 SELECT * FROM "Airbnb_reviews"
2 ORDER BY random()
3 limit 20;
```

Note: limit controls how many rows are returned

2.12 Foreign Keys

A **foreign key** is a cross-table constraint used for linking two tables (collection of attributes in one table that refers to the PRIMARY KEY in another table). **Referential integrity** is the constraint associated with foreign keys references

Defining Foreign Key in SQL

```
1 CREATE TABLE customers (
2     customer_id INT GENERATED ALWAYS AS IDENTITY,
3     customer_name VARCHAR(255) NOT NULL,
4     PRIMARY KEY(customer_id)
5 );
6
7 CREATE TABLE contacts(
8     contact_id INT GENERATED ALWAYS AS IDENTITY,
9     contact_name VARCHAR(255) NOT NULL,
10    phone VARCHAR(15),
11    email VARCHAR(100),
12    PRIMARY KEY(contact_id),
13    CONSTRAINT fk_customer FOREIGN KEY(customer_id) REFERENCES customers(customer_id)
14 );
```

Actions Postgres can take to maintain consistency

- NO ACTION: rejects operation if dependent rows exist (default)
- RESTRICT: Same as NO ACTION, but checked immediately (not deferred)
- CASCADE: Propagates the change automatically
- SET NULL: Sets the foreign key column to NULL
- SET DEFAULT: Sets the foreign key column to its DEFAULT value

Example: delete all related enrollments if a student or course is deleted

```
1 CREATE TABLE enroll(
2     studentif INT REFERENCES student(studentid) ON DELETE CASCADE,
3     courseid TEXT REFERENCES course(courseid) ON DELETE CASCADE,
4     semester TEXT,
5     grade CHAR(2),
6     PRIMARY KEY (studentid, courseid, semester)
7 );
```

2.13 Joins

2.13.1 Inner Join

The following two queries are an example of how we can join tables. (Note: inner join can be replaced with just join since inner join is the default join)

```
1 SELECT N.MFSymbol, top10_holding
2 FROM mutualfunds M, mfnews N
3 WHERE M.fund_symbol = N.MFSymbol AND N.src ilike '%marketwatch%';
4
5 SELECT N.MFSymbol, top10_holding
6 FROM mutualfunds M
7 JOIN mfnews N ON M.fund_symbol = N.MFSymbol
8 WHERE N.src ilike '%marketwatch%';
```

The query processor of the DBMS usually evaluates the selection predicates before the join predicates as a join is expensive

2.13.2 Left Outer Join

Contains all records between left and right table that meet join condition and any records in left table that don't meet join condition. Rows for which the join condition are not met have the columns in the right table padded with nulls.

```
1 SELECT DISTINCT fund_symbol, newsid
2 FROM mutualfunds M LEFT JOIN mfnews N on M.fund_symbol = N.MFSymbol
3 ORDER BY newsid NULLS last;
```

Nulls can't be ordered but they can be placed at beginning or end. Default is to keep them as is.

2.13.3 Right Outer Join

Contains all records between left and right table that meet join condition and any records in right table that don't meet join condition. Rows for which the join condition are not met have the columns in the left table padded with nulls.

```
1 SELECT DISTINCT fund_symbol, newsid
2 FROM mutualfunds M RIGHT JOIN mfnews N on M.fund_symbol = N.MFSymbol
3 ORDER BY newsid NULLS last;
```

2.13.4 Full Outer Join

Contains all records between left and right table that meet join condition and any records in left or right table that don't meet join condition. Rows for which the join condition are not met have the columns in the other table padded with nulls.

```
1 SELECT DISTINCT fund_symbol, newsid
2 FROM mutualfunds M FULL JOIN mfnews N on M.fund_symbol = N.MFSymbol
3 ORDER BY newsid NULLS last;
```

2.13.5 Natural Join

Combines tuples from two tables by matching on common attributes. There is also a natural left outer join, natural right outer join, and natural full outer join

```
1 SELECT director
2 FROM movie NATURAL JOIN schedule
3 WHERE theater = 'Hillcrest';
```

2.14 Nested Queries

We can place sub queries within a query

```
1 SELECT mfsymbol, title, management_name
2 FROM mutualfunds m LEFT JOIN (
3     SELECT DISTINCT n.MFSymbol, u.title
4     FROM mfnes n, usnewspaper u
5     WHERE n.newsId = u.id) a ON m.fund_symbol = a.MFSymbol;
```

The same query can be written with the **WITH** clause (longevity of table is within the query)

```
1 WITH miniquery AS (
2     SELECT DISTINCT n.MFSymbol, u.title
3     FROM mfnes n, usnewspaper u
4     WHERE n.newsId = u.id)
5 SELECT MFSymbol, title, management_name
6 FROM mutualfunds m LEFT JOIN miniquery on m.fund_symbol = miniquery.MFSymbol;
```

EXISTS represents an existential quantification that checks if there is at least one tuple satisfying a specific predicate (can also do NOT EXISTS)

```
1 SELECT DISTINCT name
2 FROM "Airbnb_listings" X
3 WHERE exists (
4     SELECT Ar.listing_id
5     FROM "Airbnb_reviews" Ar
6     WHERE X.id = Ar.listing_id
7 );
```

2.15 Null Values

- We can test if an attribute A is null by doing *ISNULL* or *ISNOTNULL*
- Arithmetic operations involving null return a null value
- Comparisons involving null return unknown (new truth value)
- Boolean operations with true, false, unknown (WHERE clause conditions evaluating to unknown are treated as false)

AND			OR		
true	unknown	unknown	true	unknown	true
false	unknown	false	false	unknown	unknown
unknown	unknown	unknown	unknown	unknown	unknown
NOT					
unknown			unknown		

- All aggregate operations except count(*) ignore tuples with null values on the aggregated attributes
- Null group-by attributes are treated like any other value

2.16 Views

A view is a "virtual" table that is derived from other tables. The table is not physically stored unless it is a materialized view and it allows full query operations.

```
1  -- Creating a view
2  CREATE VIEW works_on_new AS
3  SELECT fname, lname, pname, hours
4  FROM employee, project, works_on
5  WHERE ssn=essn AND pno=pnumber
6  GROUP BY pname1;
7  -- Query on view
8  SELECT fname, lname
9  FROM works_on_new
10 WHERE pname='Seena';
11 -- Drop when no longer needed
12 DROP works_on_new;
```

Materialized views persists the result in table-like form so there is no need to compute the result when a query is issued. Enabled faster access to data but the view needs to be refreshed to ensure data is up to date.

```
1  -- Creating a materialized view
2  CREATE MATERIALIZED VIEW v AS
3  SELECT device_name, pid, price
4  FROM devices d
5  JOIN parts p ON d.pid = p.pid;
6  -- Refreshing materialized view
7  REFRESH MATERIALIZED VIEW v;
8  -- Refresh materialized view concurrently to prevent locking out concurrent selects
9  REFRESH MATERIALIZED VIEW CONCURRENTLY v;
```

2.17 Regular Expressions

- The `~` operator regex tests value on left against regex on right and `~ *` does the same just case insensitively
 - Does an address string contain something like "Avenue"?
`SELECT '100 Byron Avenue' ~ 'Avenue$'`
 - Does a string start with the letter "T", any case
`SELECT 'theorem' ~ '^T'`
 - Does a string contain digits to form a North American phone number (3 for area code, 3 for exchange, 4 for local)?
`SELECT '(416) 555-1212' ~ '^\D*\d{3}\D*\d{3}\D*\d{4}\D*$'`
- Regex Expressions
 - `.` matches any character
 - `\s` matches "empty space" characters like space and tab
 - `\S` is the opposite of `\s`, so it matches anything that *isn't* a space
 - `\d` matches any number character, and `\D` does the opposite
 - `\w` matches any "word" character (a-z, 0-9), and `\W` does the opposite
 - `^` binds the pattern to the start of the input
 - `$` binds the pattern to the end of the input
 - `()` marks out portions of the pattern as "matches" to be made available for further processing
 - `*` matches 0–N repetitions of the character preceding it
 - `+` matches 1–N repetitions of the character preceding it

- {N} matches N repetitions of the character preceding it
- [] A bracket expression. Matches a single character that is contained within the brackets.
- [abc] matches "a", "b", or "c", and [a-z] specifies a range that matches any lowercase letter from "a" to "z".
- [^] Matches a single character that is *not* contained within the brackets.
- ^A+ would match strings that start with one or more 'A' characters
- \d+ would match any combination of one or more digits in order
- ^\S would match any string that did not start with white space

2.17.1 Regexp_match

Returns array based on first match to regex where each array element is what is inside the capture group denoted by parentheses

```
1 WITH regex AS (
2     SELECT regexp_match('(416) 555-1212', '^D*(\d{3})\D*(\d{3})\D*(\d{4})\D*$') AS match
3 )
4 SELECT match[1] AS area_code, match[2] AS exchange, match[3] AS local
5 FROM regex;
```

2.17.2 Regexp_matches

Returns a set of text arrays of captures sub strings. 0 rows if no match, 1 row if there is a match and the g flag is no set, N rows if there are N matches and the g flag is set

```
1 SELECT redexp_matches('foobarbequebazilbarfbonk', '(b[^b]+)(b[^b]+)', 'g')
2 -- {bar, bequeue}, {bazil, barf}
3 SELECT redexp_matches('foobarbequebazilbarfbonk', '(b[^b]+)(b[^b]+)')
4 -- {bar, bequeue}
```

2.17.3 Regexp_split

Splits a string into a collection of sub strings based on regex parameter as separator

```
1 SELECT trim(regexp_split_to_table(neighborhood, E'[ ,]')) as place FROM "Airbnb_listings";
```

2.18 Arrays

2.18.1 Extracting Elements

```
1 SELECT DISTINCT amenities[1] FROM "Airbnb_listings" ORDER BY amenities[1];
2
3 SELECT DISTINCT amenities[1:3] FROM "Airbnb_listings" ORDER BY amenities[1:3];
```

2.18.2 Find Empty Array

Since an empty array is not null, we can use array_length to check if an array is empty since it returns null on empty arrays. (Second argument in array_length function is dimension)

```
1 SELECT id, amenities
2 FROM "Airbnb_listings"
3 WHERE array_length(amenities, 1) IS NULL;
```

2.18.3 Select Based on Array Content

```
1 SELECT id, amenities
2 FROM "Airbnb_listings"
3 WHERE 'Carbon Monoxide Detector' = any(amenities);
```

2.18.4 Unnest

Creates a row for each item in list

```
1 SELECT DISTINCT id, unnest(amenities) as am
2 FROM "Airbnb_listings"
3 ORDER BY id;
```

If we unnest multiple array columns, the unnesting will be done in order and there are no "cross" tuples. (null filled in when arrays don't have equal length)

2.18.5 Contains

Used to check if multiple values occur in array

```
1 SELECT id, amenities
2 FROM "Airbnb_listings"
3 WHERE amenities @> ARRAY['Cable TV','Carbon Monoxide Detector','Doorman'];
```

The @ > operator eliminates duplicates so ARRAY[1,4,3] @ > ARRAY[3,1,3] evaluates to true.

3 Conceptual Modeling

3.1 Entities

- "objects" or "things of interest" which stand alone with their own properties
- Distinguishable from other objects and things of interest
- Have a unique name in the schema
- In a large-scale problem, there can be hundreds of entities

Examples:

- University Schema: Student, Faculty, Courses, Classroom, ...
- Hospital Schema: Doctors, Patients, Nurses, Diseases, Procedures, Medicines, ...

3.2 Attributes

- Properties of entities (an entity can have many attributes)
- Attribute Types
 - Single Value
 - Composite Value (ex. First, Middle, and Last name as one attribute). Split into single-values attributes to query at component level.
 - Multiple Value (ex. telephone numbers)
 - Nested Values (ex. address may have street number, street name, apt number)

Examples:

- Student: student_id, Name, Age, Year_of_study, Current_GPA, Phone_number, ...

3.3 Relationships

- Captures the associations between entities
- Captures how one row in a table (entity) is related to other rows in another table
- Captures as constraints in a database
- Relationships can also have properties
- A relationship can be between two or more entities

Examples:

- A student can take one or more courses
- A course can be taken by one or more students
- A course is taught by one instructor
- A student is given only one grade for each course taker by the student

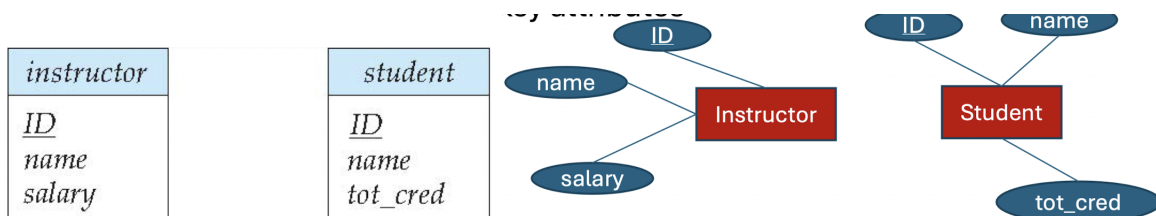
Types of Relationships:

- By Cardinality
 - One-to-one relationship (ex. A student and PID)
 - One-to-many relationship (ex. An instructor can have one or more emails)
 - Many-to-one relationship (ex. Many students belong to a college)
 - Many-to-many relationship (ex. Student-Course relationship)
- By Number of Entities Involved
 - Binary (ex. student,course)
 - Ternary (ex. student,course,instructor)
 - N-ary (student,course,instructor,...)

3.4 Entity-Relationship Model Diagram

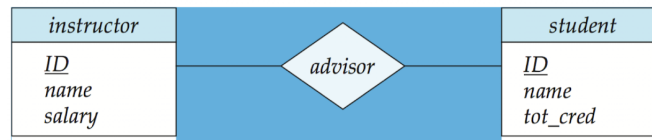
3.4.1 Entity Sets

- Rectangles can be used to represent entity sets
- Attributes are listed inside entity rectangle (or as ellipses connected to the entity set)
- Underline indicated primary key attributes

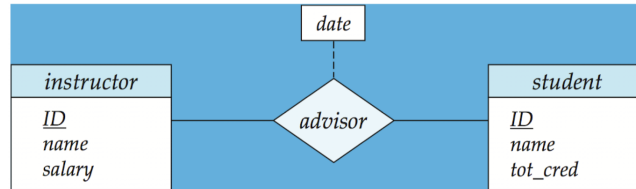


3.4.2 Relationship Sets

- Diamonds represent relationship sets

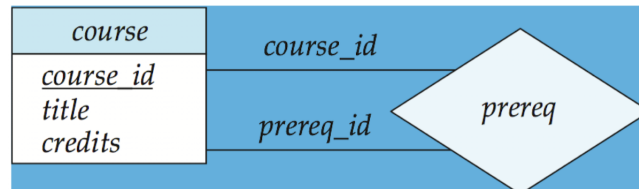


- Attributes can also be added to relationship sets



3.4.3 Roles

- entity sets of a relationship don't need to be distinct (each occurrence of an entity set plays a "role" in the relationship)
- The labels "course_id" and "prereq_id" in the diagram below are called roles
- Some other examples of role relationships are parent_of person entity, sub_folder of folder entity, and manager_of employee entity

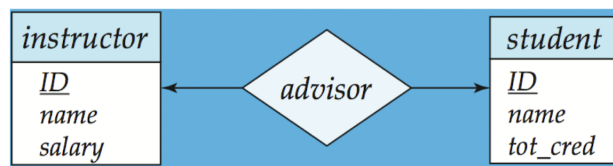


3.4.4 Cardinality Constraints

- A directed line ($\rightarrow$) is used to signify "one"
- An undirected line (-) is used to signify "many"

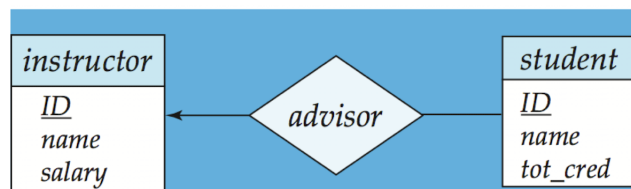
One-to-One

A student is associated with at most one instructor via advisor



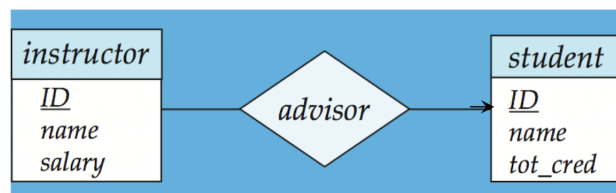
One-to-Many

An instructor is associated with several (including 0) students via advisor and a student is associated with at most one instructor via advisor



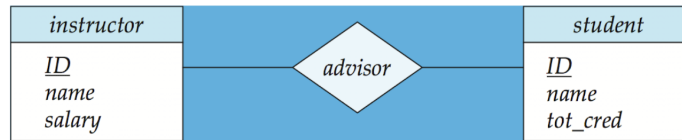
Many-to-One

An instructor is associated with at most student via advisor and a student is associated with several (including 0) instructors via advisor



Many-to-Many

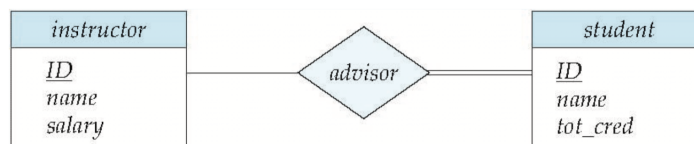
An instructor is associated with several (possibly 0) students via advisor and a student is associated with several (possibly 0) instructors via advisor



3.4.5 Total and Partial Participation

- A double line indicates total participation which means every entity in the entity set participates in at least one relationship in the relationship set
- Partial participation is when some entities may not participate in an relationship in the relationship set

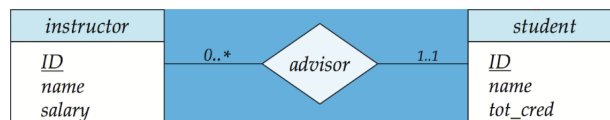
The diagram below shows every student must have an associated instructor in the advisor relation and participation of instructor in advisor is partial



3.4.6 More Complex Constraints

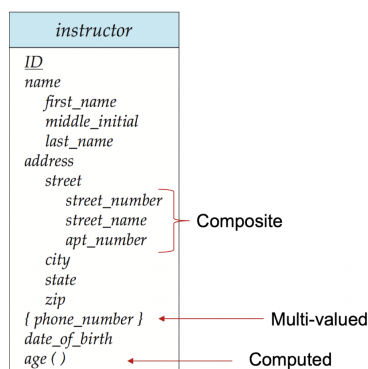
- A line can have an associated minimum, maximum cardinality shown in the form $l..h$, where l is the minimum and h the maximum cardinality
- Minimum value of 1 indicates total participation
- Maximum value of 1 indicates that the entity participates in at most one relationship
- A maximum value of * indicates no limit

The diagram below shows that an instructor can advise 0 or more students and a student must have 1 advisor (student also can't have multiple advisors)



3.4.7 Entities With Complex Attributes

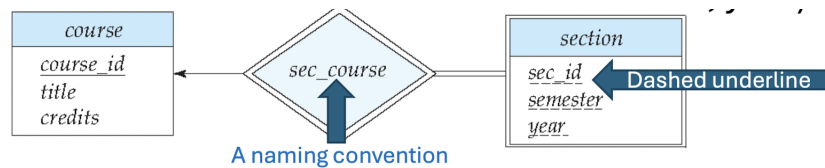
The diagram below illustrates how entities can be shown with complex attributes



3.4.8 Weak Entities

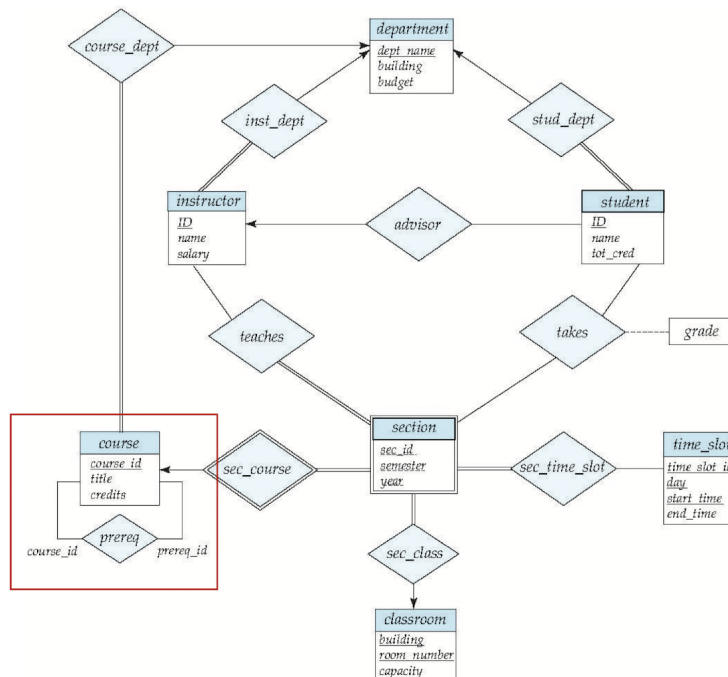
- Normally entities have keys - at least one key called primary key
- A Weak entity has no key (or an insufficiently identifying key) and 'borrows' key from a strong (normal) entity (ex. A class with classNum, term, year, startDate, endDate, maxSize also needs Course.id to make it useful)
- A weak entity "hangs off" the strong entity and this relationship between a strong and weak entity is called an identifying relationship. (primary key of weak entity contains primary key of the strong entity along with other attributes in it called discriminators)
- In ER Diagrams, a weak entity set is shown with a double rectangle
- Discriminators of a weak entity set are underlined with a dashed line
- The relationship set connecting the weak entity to the identifying strong entity set is depicted by a double diamond

In diagram below, the primary key for section is (course.id, sec.id, semester, year)



3.4.9 Complete Example

Below is a ER Diagram for a University Enterprise



4 Relation Database Design Dependencies

Reference Table for This Section

S#	SNAME	SCITY	P#	PNAME	PCITY	QTY
S1	Smith	London	P1	Nut	London	100
S1	Smith	London	P2	Bolt	Paris	150
S2	Adams	Paris	P1	Nut	London	200
S3	Blake	Rome	P3	Screw	Rome	250
S4	Clark	Paris	P2	Bolt	Paris	300

4.1 Anomalies

- **Insertion Anomaly**

- Can't insert information about an entity without inserting fake information in the table
- For example, if you want to add a new supplier S5 (Jones,Berlin) that hasn't made a shipment, we can't since the P# and QTY can't be null. We would have to insert a fake shipment (QTY=0)

- **Deletion Anomaly**

- Deleting information destroys unrelated information
- For example, if we delete the last shipment of a supplier, we end up losing all record of the supplier

- **Update Anomaly**

- Data redundancies can cause inconsistent updates since we have to make sure we update every redundant piece of information
- For example, if Smith moves from London to Berlin, we must update every row where S#=S1

4.2 Guidelines For Design

1. Each tuple (row) in a relation (table) should represent one entity or relationship instance
 - Attributes of different entities shouldn't be mixed in the same relation
 - Only foreign keys should be used to refer to other entities
 - Entity and relationship attributes should be kept apart as much as possible
2. Design a schema that does not suffer from insertion, deletion, and update anomalies
3. Reduce null values
 - It is possible that a relation can be created with many attributes in which only some apply to each entity causing many null values
 - Null columns should be exceptions rather than the rule
 - Attributes that are frequently null can be placed in separate relations (with primary key)
4. Spurious Tuples
 - Unwanted rows that can appear in joins
 - The relations should be designed to satisfy lossless join condition which means no spurious tuples should be generated by going a natural join of any relations

4.3 Functional Dependency

- Let R be a relation and X and Y be two sets of attributes in that relation $X \rightarrow Y$ (X determines Y) if and only if for any two tuples in R whose X values agree implies that their Y values also agree. We can say Y is functionally dependent on X or X functionally determines Y
- $X \rightarrow Y$ in R specifies a constraint on all relation instances $r(R)$
- FDs are derived from the real-world constraints on attributes

4.3.1 Keys

- Keys are a special kind of FD
- If K is a key of R , the K functionally determines all attributes in R
- For every key K of relation R , there is an FD $K \rightarrow R$
- A superkey is a set of attributes S subset of R with the property that no two tuples $t1$ and $t2$ in any legal relation state r of R will have $t1[S] = t2[S]$
- A key K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey anymore (A key is a minimal superkey)

4.3.2 Inference Rules for FDs

- Armstrong's Axioms
 1. (Reflexive) If Y is a subset of X then $X \rightarrow Y$
 2. (Augmentation) If $X \rightarrow Y$, then $XZ \rightarrow YZ$
 3. (Transitive) If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$
- Union: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
- Decomposition: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
- Closure of a set F of FDs is the set F^+ of all FDs that can be inferred from F
- Closure of a set of attributes X with respect to F is the set X^+ of all attributes that are functionally determined by X (Can be calculated by applying axioms using FDs in F)
 - Computing the closure of a set of FDs can be expensive
 - Typically if we want to check if $X \rightarrow Y$ is in the closure set of FDs F , we can compute the attribute closure of X (denoted by X^+)
 - Example: To see if $F\{A \rightarrow B, B \rightarrow C, CD \rightarrow E\}$ implies $A \rightarrow E$, we can check if E is in A^+
- Two sets of FDs F and G are equivalent if
 - Every FD in F can be inferred from G
 - Every FD in G can be inferred from F
 - Hence, F and G are equivalent if $F^+ = G^+$
- A set of FDs F covers G if
 - Every FD in G can be inferred from F
 - If F and G are equivalent, they both cover each other
- A set of FDs is a minimal set of FDs if

- Every dependency in F has a single attribute for its RHS
- We cannot remove any dependency from F and have a set of dependencies that is equivalent to F
- We cannot replace any dependency $X \rightarrow Y$ in F with a dependency $Y \rightarrow A$, where Y is a proper subset of X and still have a set of dependencies that is equivalent to F
- A minimal set of FDs is also called the cononical cover denoted as F_C

4.4 Proving Losslessness

4.4.1 Chase Algorithm

1. **Unfold Projections:** Replace each projection in the decomposition join by an R-atom with fresh variables in the non-projected attributes
2. **Apply functional dependencies:** For each FD $X \rightarrow Y$, identify all atoms that agree on the attributes of X and make the corresponding attributes in Y equal across those atoms
3. **Remove redundant atoms:** After enforcing FDs, if one atom's attributes are all equal to the head variables (every attribute is determined), then the atom represents the whole R-tuple. Drop other atoms and local variables that contribute no new attributes
4. **Read off the minimized query:** The remaining atom(s) for the minimized conjunctive query and if the body reduces to a single R-atom $R(\text{head vars})$, the decomposition is lossless join/

4.4.2 Example

The decomposition join query is

$$Q(s, sn, sc, p, pm, pc, q) : - S(s, sn, sc), P(p, pm, pc), SP(s, p, q)$$

and by definition of the projections:

$$S(s, sn, sc) : - R(s, sn, sc, x_4, x_5, x_6, x_7)$$

$$P(p, pm, pc) : - R(x_8, x_9, x_{10}, p, pm, pc, x_{11})$$

$$SP(s, p, q) : - R(s, x_{12}, x_{13}, p, x_{14}, x_{15}, q)$$

so unfolded:

$$Q : - R(s, sn, sc, x_4, x_5, x_6, x_7), R(x_8, x_9, x_{10}, p, pm, pc, x_{11}), R(s, x_{12}, x_{13}, p, x_{14}, x_{15}, q)$$

Let's rewrite the body attributes explicitly:

Atom	S#	SNAME	SCITY	P#	PNAME	PCITY	QTY
R_1	s	sn	sc	x_4	x_5	x_6	x_7
R_2	x_8	x_9	x_{10}	p	pm	pc	x_{11}
R_3	s	x_{12}	x_{13}	p	x_{14}	x_{15}	q

1) Initial table (before applying FDs)

Row \ Attr	S#	SNAME	SCITY	P#	PNAME	PCITY	QTY
R_1 - from S	a_s	a_{sn}	a_{sc}	b_p	b_p	b_p	b_7
R_2 - from P	b_s	b_p	b_{10}	a_p	a_{pm}	a_{pc}	b_{11}
R_3 - from SP	a_s	b_{12}	b_{13}	a_p	b_{14}	b_{15}	a_q

2) Apply FD $S\# \rightarrow SNAME, SCITY$

Rows with the same $S\#$ must agree on $SNAME, SCITY$.

R_1 and R_3 share $[a_s]$, so unify:

- $b_{12} \Rightarrow a_{sn}$
- $b_{13} \Rightarrow a_{sc}$

Row \ Attr	S#	SNAME	SCITY	P#	PNAME	PCITY	QTY
R_1	a_s	a_{sn}	a_{sc}	b_p	b_p	b_p	b_7
R_2	b_s	b_p	b_{10}	a_p	a_{pm}	a_{pc}	b_{11}
R_3	a_s	a_{sn}	a_{sc}	a_p	b_{14}	b_{15}	a_q

3) Apply FD $P\# \rightarrow PNAME, PCITY$

Rows with the same $P\#$ must agree on $PNAME, PCITY$.

R_2 and R_3 share $[a_p]$, so unify:

- $b_{14} \Rightarrow a_{pm}$
- $b_{15} \Rightarrow a_{pc}$

Row \ Attr	S#	SNAME	SCITY	P#	PNAME	PCITY	QTY
R_1	a_s	a_{sn}	a_{sc}	b_p	b_p	b_p	b_7
R_2	b_s	b_p	b_{10}	a_p	a_{pm}	a_{pc}	b_{11}
R_3	a_s	a_{sn}	a_{sc}	a_p	a_{pm}	a_{pc}	a_q

4) Apply FD $S\# \rightarrow QTY$

Rows that agree on $(S\#, P\#)$ must agree on QTY.
 R_3 already has (a_s, a_p) and a_q .
 Unify R_1 's and R_2 's unknown quantities (if needed) to a_q via agreement with R_3 on $S\#$ or $P\#$ and via the third atom that ties both together:

- $b_7 \Rightarrow a_q$ (via $R_1 + R_3$)
- $b_{11} \Rightarrow a_q$ (via $R_2 + R_3$)

Row \ Attr	S#	SNAME	SCITY	P#	PNAME	PCITY	QTY
R_1	a_s	a_{sn}	a_{sc}	b_4	b_5	b_6	a_q
R_2	b_8	b_9	b_{10}	a_p	a_{pn}	a_{pc}	a_q
R_3	a_s	a_{sn}	a_{sc}	a_p	a_{pn}	a_{pc}	a_q

After step 4, the result is same as the original relation so it is lossless

5 Relation Database Design Normalization

- If a relation is in a certain normal form, it is known that certain kinds of problems are avoided/minimized
- Normalization is carried out in practice so that the resulting designs are of high quality and meet desirable properties
- Additional Vocab
 - **Prime Attribute:** Attribute that is a member of some candidate key
 - **Nonprime Attribute:** Not a prime attribute (not a member of candidate key)
 - **Full Functional Dependency:** An FD $Y \rightarrow Z$ is a FFD where the removal of any attribute from Y means the FD doesn't hold anymore
 - **Transitive Functional Dependency:** An FD $X \rightarrow Z$ that can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$
 - **Nontrivial FD:** A FD $X \rightarrow Y$ where Y is not contained in R

5.1 First Normal Form

- Every cell in the table is atomic
- No duplicate rows
- Every cell has a value (no nulls)

(a)

EMP_PROJ			Projs	
Ssn	Ename	Pnumber	Hours	

(b)

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

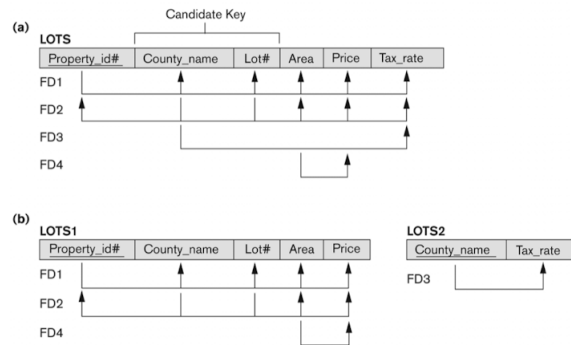
EMP_PROJ1	
Ssn	Ename

EMP_PROJ2		
Ssn	Pnumber	Hours

Figure 10.9
 Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a nested relation attribute PROJ.S. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

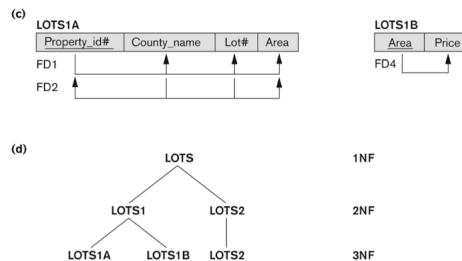
5.2 Second Normal Form

- Every non-prime attribute is fully functionally dependent on every candidate key
- 2NF Decomposition Process
 - Look at every FD in relation R
 - If the LHS is a subset of any candidate key for R then create a relation with LHS and RHS of that key and create a relation by removing RHS attributes from R



5.3 Third Normal Form

- A relation schema is in 3nf if it is in 2nf and no non-prime attribute A in R is transitively dependent on any candidate key
- 2nf to 3nf process
 - For each functional dependency $\alpha \rightarrow \beta$, if α is not a superkey we need to verify if each attribute in β is contained in a candidate key of R



5.4 Boyce-Codd Normal Form

- Whenever $X \rightarrow Y$ is a nontrivial FD that holds in R , X is a superkey
- Every relation that is in BCNF is also in 3NF
- BCNF Decomposition Process
 - Look among the given FD's for a BCNF violation $X \rightarrow Y$
 - If any FD following from F violates BCNF, then there will surely be an FD in F itself that violates BCNF
 - Compute attribute closure of X^+
 - Replace R by relations with schemas $R_1 = X^+$ and $R_2 = R - (X^+ - X)$

Drinkers(name, addr, beersLiked, manf, favBeer)

- $F = \text{name} \rightarrow \text{addr}, \text{name} \rightarrow \text{favBeers}, \text{beersLiked} \rightarrow \text{manf}$
- Pick BCNF violation $\text{name} \rightarrow \text{addr}$

Close the left side:

- $\{\text{name}\}^+ = \{\text{name}, \text{addr}, \text{favBeer}\}$

Decomposed relations:

1. **Drinkers1**(name, addr, favBeer)

2. **Drinkers2**(name, beersLiked, manf)

Note: need to check if subsequent relations are in BCNF and in this case Drinkers2 violates BCNF

6 Recursive Queries

- **Fixpoint:** A fixpoint of a function T is a value X^* such that $T(X^*) = X^*$. For recursive rules, T is the immediate consequence operator that computes new facts from known ones. We iterate $R_{i+1} = T(R_i)$ until no new facts appear. The smallest such R^* is the least fixpoint (LFP)
- **Naive Evaluation Algorithm:** Apply the rules using the entire relation each iteration. It is correct but inefficient, since each iteration recomputes joins over all previously derived data causing redundant matches to be revisited and computational costs to grow rapidly
- **Semi-Naive Evaluation:** Instead of reusing the whole relation, only use new tuples discovered in the last relation. This avoids redundant work and PostgreSQL uses this internally.

Ancestor Example

```
1 WITH RECURSIVE ancestors AS (  
2   -- Base case: every parent is an ancestor at depth 1  
3   SELECT  
4     child,  
5     parent AS ancestor,  
6     1 AS depth  
7   FROM parents  
8   UNION  
9   -- Recursive step: parent's ancestors are also the child's ancestors  
10  SELECT  
11    p.child,  
12    a.ancestor,  
13    a.depth + 1 AS depth  
14  FROM parents p  
15  JOIN ancestors a  
16  ON p.parent = a.child  
17  WHERE p.child <> a.ancestor -- prevent self-cycles  
18 )  
19 SELECT DISTINCT child, ancestor, depth  
20 FROM ancestors  
21 ORDER BY child, depth;
```

Graph Reachability Example

This example uses an array to track visited locations to avoid cycles

```
1 WITH RECURSIVE reachable(origin, destination, path) AS (  
2   -- Anchor part  
3   SELECT origin, destination, ARRAY[origin, destination]  
4   FROM flights  
5   UNION ALL  
6   -- Recursive part  
7   SELECT r.origin, f.destination, path || f.destination  
8   FROM reachable r  
9   JOIN flights f ON r.destination = f.origin  
10  WHERE NOT f.destination = ANY(r.path)  
11  )  
12  SELECT origin, destination, path  
13  FROM reachable;
```

7 Logical Query Optimization

7.1 Overall Picture

1. **SQL Query:** Query gets parsed into an abstract syntax tree (string is tokenized and parsed into a tree structure)
2. **Initial Relational Algebra:** The AST is translated into a logical algebra tree where nodes represent abstract operations such as selection, projection, join, etc
3. **Optimized Algebra:** Applied equivalence rules to simplify the plan such as predicate push-down, join re-ordering, etc (goal is to preserve results but improve expected cost)
4. **Physical Plan Selection:** logical operators are mapped to concrete algorithms (Seq Scan vs Index Scan/ Nested Loop Join vs Hash Join). The resulting tree is the physical plan
5. **Execution Engine/Runtime Plan:** The physical plan's operators run, pulling tuples from child nodes. Intermediate results streamed from bottom to top using iterators. Final rows are then returned to the user or client application

7.2 Logical Rewriting

- **Key Principles**

- Equivalence Preservation: Every rewrite must preserve the exact same result as the original query
- Relational Algebra as the Internal Language: SQL is translated into algebra, and rewrites are applied at the algebra level
- Reduce Before Join: Apply filters and projects as early as possible to reduce tuple count and tuple width
- Local Transformations: Most rewrites operate on small algebra subtrees without changing the whole plan
- Iterative Refinement: rewrites occur over multiple passes until no more rules apply (fixpoint)

- **Heuristic Goals**

- Push Selections Down: minimize the number of rows flowing into join operations
- Push Projections Down: eliminate unnecessary attributes to reduce memory and join costs
- Reorder Joins: Use commutativity and associativity to form cheaper join orders
- Replace Expensive Operators with Cheaper Ones: example is converting nested sub queries into semi-joins or IN into joins

- Exploit Constraints & Keys: Use primary keys, foreign keys, uniqueness, and nullability information to simplify conditions
- Remove Redundant Operators: can combine cascaded selections, remove unnecessary projections, and eliminate cartesian products when possible

7.3 Query Rewriting

- **Commutativity Rules**

- Selection: $\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R))$
- Join: $R \bowtie S = S \bowtie R$
- Union: $R \cup S = S \cup R$
- Intersection: $R \cap S = S \cap R$

- **Associativity Rules**

- Join: $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$
- Union: $(R \cup S) \cup T = R \cup (S \cup T)$
- Intersection: $(R \cap S) \cap T = R \cap (S \cap T)$

- **Selection Rules**

- Cascade of Selections: $\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$
- Commuting Selection and Projection (projection keeps attributes used for selection): $\pi_L(\sigma_c(R)) = \sigma_c(\pi_L(R))$
- Pushing Selection Through Join (predicate refers to attributes of one relation): $\sigma_c(R \bowtie S) = (\sigma_c(R)) \bowtie S$

- **Projection Rules**

- Cascade of Projections: $\pi_{L_1}(\pi_{L_2}(R)) = \pi_{L_1}(R)$ if $L_1 \subseteq L_2$
- Projection and Join: $\pi_L(R \bowtie S) = \pi_{L_R \cup L_S}(R \bowtie S)$
- Pushing Projection Below Join (only attributes required for join and output): $\pi_L(R \bowtie S) = (\pi_{L_R}(R)) \bowtie (\pi_{L_S}(S))$

- **Join and Selection Interaction**

- Theta-Join Equivalence: $R \bowtie_c S = \sigma_c(R \times S)$
- Pushing Selections into Join Operands: $\sigma_{c_R \wedge c_S \wedge c_{RS}} = (\sigma_{c_R}(R)) \bowtie_{c_{RS}} (\sigma_{c_S}(S))$

- **Set Operation Rules**

- Selection Distributes over Union: $\sigma_c(R \cup S) = \sigma_c(R) \cup \sigma_c(S)$
- Projection Distributes over Union: $\pi_L(R \cup S) = \pi_L(R) \cup \pi_L(S)$

- **Aggregation and Grouping Rules**

- Selection Pushdown through Grouping: $\sigma_c(\gamma_G(F)(R)) = \gamma_G(F)(\sigma_c(R))$
- Projection Before Aggregation: retain only grouping and aggregate attributes

- **Rename Rules**

- $\rho_{A \rightarrow B}(\sigma_c(R)) = \sigma_{c'}(\rho_{A \rightarrow B}(R))$ where c' is c rewritten with new attribute names
- Allows for selections and projections to move freely past renaming
- Helps with join disambiguation

- **Semi-Join**

- A semi-join between relations R and S written $R \ltimes S$ returns all tuples of R that have at least one matching tuple in S
- Produces fewer attributes than a normal join (only R attributes)
- Used for equivalence checks, not full tuple pairing (IN, EXISTS, WHERE R.key IN S.key)
- Replaces expensive nested sub queries with set-based operators

Query Rewriting Example

Original Query

```

1 SELECT s.sname
2 FROM Student s
3 WHERE s.major = 'CS'
4     AND EXISTS (
5         SELECT 1
6         FROM Enrollment e
7         JOIN Course c ON e.cid = c.cid
8         WHERE e.sid = s.sid
9             AND c.dept = 'CS'
10    );

```

Initial Logical Plan:

```

1 PROJ d.dname, e.name, p.pid (
2     SELECT d.location = 'LA'
3     AND e.salary > 80000
4     AND p.budget >= 100000
5     AND p.budget <= 300000
6
7     (
8         (Department d THETA_JOIN d.deptid = p.deptid Project p)
9         THETA_JOIN p.pid = a.pid Assignment a
10        THETA_JOIN a.empid = e.empid Employee e
11    )
12 )

```

Optimized Logical Plane

```

1 PROJ d.dname, e.name, p.pid (
2     (
3         PROJ d.dname, d.deptid (
4             SELECT d.location = 'LA' (Department d)
5         )
6         THETA_JOIN d.deptid = p.deptid
7         PROJ p.pid, p.deptid (
8             SELECT p.budget >= 100000 AND p.budget <= 300000 (Project p)
9         )
10    )
11    THETA_JOIN p.pid = a.pid
12    PROJ a.pid, a.empid (
13        Assignment a
14    )
15    THETA_JOIN a.empid = e.empid
16    PROJ e.name, e.empid (
17        SELECT e.salary > 80000 (Employee e)
18    )
19 )

```

8 Physical Query Optimization

A physical query plan is a tree of concrete low-level database operators (scans, joins, sorts, hashing, index lookups) the DBMS will run to execute an SQL query. A physical query plan is a rooted tree where leaves are access methods (SeqScan, IndexScan, BitmapScan) and internal nodes are physical operators for implementing relational algebra (HashJoin, MergeJoin, Sort, Hash Aggregate, etc)

8.1 Physical Properties of Operator

- **Output Ordering**
 - Are results ordered? (ex. sort by dept or salary)
 - Are results clustered? (ex. from B+ tree index scan)
 - Does the operator preserve or destroy sort order?
 - merge join preserves order while hash join destroys it
- **Partitioning/Hashing**
 - Are tuples partitioned by a hash function? (ex. hash joins build side produces hash buckets)
 - Can the next operator reuse this partitioning (ex. hash aggregation can reuse hash partitions from a hash join)
- **Uniqueness**
 - Are duplicates preserved?
 - Does the operator guarantee unique keys?
- **Tuple Format/Row Layout**
 - Column order
 - Presence of "system columns"
 - Whether tuples are materialized or streamed
 - Whether the operator keeps later materialization (ex. if index returns TIDs only)

8.2 Some Physical Operators

- **Access Paths**
 - Sequential Scan: big scans, low selectivity
 - Index Scan: selective lookups
 - Index-Only Scan: avoid heap I/O
 - Bitmap Scan: combining multiple indexes
- **Join Methods**
 - Nested Loop: tiny outer
 - Index Nested Loop: index-driven join
 - Hash Join: large tables, no order
 - Merge Join: sorted inputs
 - Grace Hash Join Disk: partitioned version of hash join used for huge relations
- **Other Operators**
 - Sort Merge Union: used for union operators
 - Sort: ordering
 - Hash Aggregate: grouping by hashing
 - Materialize: caching
 - Limit: early stopping
- **Parallel Operators**
 - Gather Merge: merge-sorts outputs from parallel workers (multithreaded)
 - Parallel Seq Scan/Parallel Index Scan: parallelized access operators
 - Parallel Hash Join: parallel build + probe
 - Parallel Append: dispatch parallel scans on partitions

8.3 Cost Model

The optimizer estimates: $\text{cost} = \text{I/O cost} + \text{CPU cost} + \text{memory cost}$

- **I/O Cost**

- package seeks and reads
- Index root-to-leaf traversal
- Sequential Scan page count
- Temporary spill files (sort/hash)

- **CPU Cost**

- Predicate evaluation
- Hashing tuples
- Comparisons
- Join condition evaluation
- Function evaluation

- **Memory Cost**

- Hash table size
- Sort buffer requirements
- Whether spill-to-disk is necessary

8.4 Cardinality Estimation

Optimizer must estimate the following

- **Selectivity of each predicate:** Using histograms, MCVs, min/max, null fraction to estimate how many results will be filtered out with predicate (ex. use histogram to estimate percent of rows filtered with $\text{age} > 40$)
- **Output size of joins:** $|R \bowtie S| = \frac{|R| \cdot |S|}{\max(\text{NDV}_R(A), \text{NDV}_S(A))}$ (NDV is number of distinct values)

8.5 Statistics Used by the Optimizer

- **Base Statistics**

- Total row count
- Total page count
- Tuple width
- Dead vs. live rows (dead rows inflate table size so optimizer thinks more overhead to process. VACUUM updates catalog statistics)

- **Columns Statistics**

- NDV (number distinct values)
- Histogram of value distribution
- Most common values (values + frequencies)
- Null fraction
- Correlation between column and physical order (clustering)

- **Index Statistics**

- Tree height (shorter tree means faster lookup)
- Leaf page count (number of pages that contain actual data entries)
- Fraction of duplicates (index selectivity)
- Correlation with heap order (measures how closely index order matches physical order of table. Index on date column in append only table would have high correlation while index on last name in random insert pattern will have poor correlation and index scan should be avoided)

- **Multivariate Statistics (PostgreSQL extended)**

- Multi-common most common value (MCV) lists
- Multi-column histograms (ex. city,zip_code would show correlations)
- Functional dependencies (can be learned)

8.6 Access Plan Selection

- **Sequential Scan**

- Cost=table_pages
- used when predicate not selective, table small, no matching index, most rows likely to qualify

- **Index Scan**

- Cost= $O(\log N)$ for tree walk + # of tuples $\times$ random I/O
- food for small ranges, equality lookups, moderate selectivity

- **Index-Only Scan**

- If all columns come from index
- bypass heap entirely
- best case scenario

- **Bitmap Index Scan**

- Combine multiple index conditions
- Good for OR conditions and low selectivity ranges

8.7 Data Layout

DBMS utilizes slotted pages to layout data

- **Core Components**

- Page Header: contains metadata about the page (page ID, free space pointers, record count)
- Data Records: variable length tuples containing the actual data
- Slot Directory/Line Pointer Array: collection of pointers that map to the physical location of records

- **Structure Organization**

- Records grow from the top of the page downwards
- Slot directory/line pointers grow from the bottom upward
- Free space exists in the middle between these two area

- **Slot Directory Entry Composition**

- Offset (pointer to record's physical location)
- Length (size of record in bytes)
- Optional status flag (indicating deleted/active records)

- **Slotted Page Behavior**

- Deleting Tuple: slot entry marked dead and dead tuple space is reclaimed by VACUUM (clear fragmented portions for large free space and fix poor estimates of optimizer)
- Updating Tuple: new version inserted if space allows (hot update) otherwise moved to another page and old version is dead
- In-page movement: does not break indexes since indexes point to TIDs (Block#,Offset) not by byte location

9 Indexing

- Index file store entries of a field value and pointer to record
- Dense Index: has an index entry for every search key value (hence every record) in the data file
- Sparse Index: has index entries for only some of the search values

9.1 Single Level Indexes

- **Primary Index**

- Defined on ordered data file
- The data file is ordered on a key field
- Includes one index entry for each block in the data file (index entry has key field value for first record in block which is called block anchor)
- A similar scheme can use the last record in a block
- A primary index is a sparse index since it includes an entry for each disk block of the data file and keys of its anchor record rather than for every search value

- **Clustering Index**

- Defined on ordered data file
- The data file is ordered on a non-key field unlike primary index which requires that the ordering field of the data file have a distinct value for each record
- Has one entry for each distinct value of the field (index entry points to first data block that contains records with that field value)
- It is another example of nondense index where Insertion and Deletion is relatively straightforward with a clustering index

- **Secondary Index**

- Provides a secondary means of accessing a file for which some primary access already exists
- May be on a field which is a candidate key and has a unique value in every record or a non-key with duplicate values
- Ordered file with two fields
 1. First field is of same data type as some non-ordering field of the data file that is an indexing field
 2. Second field is either a block pointer or record pointer
- There can be many secondary indexes (and hence, indexing fields) for the same file
- Includes one entry for each record in the data file; hence, it is a dense matrix

9.2 Multi-Level Indexes

- Because a single-level index is an ordered file, we can create a primary index to the index itself (index to the index is called the second-level index)
- We can repeat process to get more levels until all entries fit on singular disk block
- Multi-level index can be created as long as the first level index consists of more than one disk block

9.3 B and B+ Trees

- Most multi level indexes use B-tree and B+-tree because of insertion and deletion problem (leaves space in each tree node to allow for new index entries)
- Allow for efficient insertion and deletion of new search values
- Each node corresponds to a disk block
- Each node is kept between half-full and completely full
- **B+ Tree**
 - Used for attributes that must support range queries
 - Balanced tree
 - Every path from the root of the tree to a leaf is of the same length
 - Each nonleaf node of the tree has between $n/2$ and n children (n is fixed for particular tree)
 - It contains index pages and data pages
 - The capacity of a leaf has to be 50% or more (ex. if $n=4$ then the key for each node is between 2 to 4 and the index page will be $4+1=5$)
- B tree algorithms

Data Page Full	Index Page Full	Action
NO	NO	Place the record in sorted position in the appropriate leaf page
YES	NO	1. Split the leaf page 2. Place Middle Key in the index page in sorted order. 3. Left leaf page contains records with keys below the middle key. 4. Right leaf page contains records with keys equal to or greater than the middle key.
YES	YES	1. Split the leaf page. 2. Records with keys < middle key go to the left leaf page. 3. Records with keys >= middle key go to the right leaf page. Split the index page. 4. Keys < middle key go to the left index page. 5. Keys > middle key go to the right index page. 6. The middle key goes to the next (higher level) index. IF the next level index page is full, continue splitting the index pages.

Data Page Below Fill Factor	Index Page Below Fill Factor	Action
NO	NO	Delete the record from the leaf page. Arrange keys in ascending order to fill void. If the key of the deleted record appears in the index page, use the next key to replace it.
YES	NO	Combine the leaf page and its sibling. Change the index page to reflect the change.
YES	YES	1. Combine the leaf page and its sibling. 2. Adjust the index page to reflect the change. 3. Combine the index page with its sibling. Continue combining index pages until you reach a page with the correct fill factor or you reach the root page.

- Max number of keys is n^h
- Min number of keys is $2(n/2)^{h-1}$
- B+ tree of order 200
 - Leaves can contain up to 199 keys
 - Assuming that root has at least 100 children
 - * A 2 level B+ tree that meets these assumptions can store about 9,900 records (100 leaves each containing at least 99 keys)
 - * A 3 level B+ tree can store about 1 million keys
 - * A 4 level B+ tree can store about 100 million keys

10 Transactions

- A transaction is a unit of program execution that accessed and possibly updates various data items
- Two main issues to deal with are failures (hardware failure, system crashes, etc) and concurrent execution of multiple transactions

10.1 Basic Architecture

A typical transaction server consists of multiple processes accessing data in shared memory

- **Server Processes**
 - Receive user queries (transactions), execute them and send results back
 - Processes may be multi threaded allowing a single process to execute several user queries concurrently
 - Typically, multiple multi threaded server processes
- **Lock manager process**
 - Used for deadlock detection
- **Database writer process**
 - Output modified buffer blocks to disk continually
- **Log writer process**
 - Server processes simply add log records to log record buffer
 - Log writer process outputs log records to stable storage
- **Checkpoint process**
 - performs periodic checkpoints
- **Process monitor process**
 - Monitors other processes and takes recovery actions if any of the other processes fail
 - ex. aborting any transactions being executed by a server process and restarting it
- **Shared Memory**
 - buffer pool
 - lock table

- log buffer
- cached query plans (reused if same query submitted again)
- All database processes can access shared memory
- Databases implement mutual exclusion to prevent two processes from accessing the same data structure at the same time (semaphores/atomic instructions)
- To avoid overhead of inter process communication for lock request/grant, each database process operates directly on the lock table (lock manager process still used for deadlock detection)

10.2 Transaction States

- **Active:** the initial state that the transaction stays in while executing
- **Partially Committed:** after the final statement has been executed
- **Failed:** after the discovery that normal execution can no longer proceed
- **Aborted:** after the transaction has been rolled back and the database restored to its state prior to the start of the transaction (can restart transaction if no internal logical error or kill the transaction)
- **Committed:** after successful completion

10.3 Requirements for Transactions

- The system should ensure that updates of a partially executed transaction are not reflected in the database
- A transaction must see a consistent database
- During transaction execution the database may be temporarily inconsistent
- **ACID Properties**
 - Atomicity: Either all operations of the transaction are properly reflected in the database or none are
 - Consistency: execution of a transaction in isolation preserved the consistency of the database
 - Isolation: Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions (intermediate results must be hidden from other concurrently running transactions)
 - Durability: After a transaction completes successfully, the changes it has made to the database persist even if there are system failures

10.4 Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system (allows for increased processor and disk utilization for more throughput / reduced average response time)
- **Serializability** is a concurrency scheme where the concurrent transaction is equivalent to one that executes transactions serially
 - A schedule is a list of transactions
 - A **serial schedule** defines each transaction so they are executed without any interference from other transactions
 - A **Non-serial schedule** defines the operations from a group of concurrent transactions that are interleaved (goal is to find non-serial schedules that allow transactions to execute concurrently without interference and produce a database state that could be produced by serial execution)

- **Conflict serializability** defines two instructions on two different transactions accessing the same data item to perform a read/write operation
 - If two transactions are both read operations, they are not in conflict
 - If one transaction wants to perform a read operation and another wants to perform a write, then they are in conflict and cannot be swapped
 - If both the transactions are for write operation, then they are in conflict, but can be allowed to take place in any order because the transactions do not read the value updated by each other
- **View Serializability** is when we say two schedules are view-equivalent if the following conditions hold
 - Initial Read: Initial read of each data item in transactions must match both schedules. (If T1 reads a data item X before T2 in schedule S1 then in schedule S2, T1 should read X before T2)
 - Final Write: Final write operations on each data item must match in both the schedules. (A data item X last written by T1 in S1 then in S2, the last write operation on X should be performed by T1)
 - Update Read: If in S1, T1 is reading a data item updated by T2 then in schedule S2, T1 should read the value after the write operation of T2 on same data item.

11 Concurrency Control and Database Recovery

11.1 Lock-Based Protocol for Concurrency Control

- Controls concurrent access to a data item
- Assures that one process should not retrieve or update a record which another process is updating
- Shared Lock: the locked data items can only be read without performing modifications to it from the database
- Exclusive Lock: the locked data items can be read as well as written
- If a resource is already locked by another transaction, then a new lock request can be granted only if the mode of the requested lock is compatible with the mode of the existing lock (can have multiple shared locks but only one exclusive lock at a time)

11.1.1 Lock Structure and Operations

- Lock Control Block
 - object_id (table, page, row, tuple, etc)
 - lock_mode (S,X,IS,IX,etc)
 - holders (list of transaction ids)
 - wait_queue (transaction id, requested mode)
- Lock Modes
 - S: shared lock - allows reading
 - X: exclusive lock - allows writing
 - IS: intention to get shared locks (placed on table or page when a transaction wants to read specific rows)
 - IX: intention to get exclusive locks (placed when a transaction will modify some rows)

11.1.2 Two-Phase Locking Protocol

- First phase is when transaction begins to execute and it requires permission for the locks it needs. In second part of first phase, the transaction has obtained all the locks and the next phase starts when it first releases a lock
- Second phase is when the transaction cannot demand any new locks and can only release acquired locks
- Ensures conflict serializable schedules
- If read and write operations introduce the first unlock operation in the transaction, then it is said to be Two-Phase Locking Protocol

11.2 Timestamp-Based Protocol for Concurrency Control

- timestamp is unique identifier
- Each transaction is issued a timestamp when it enters into the system and timestamp determines serializability order
- Uses either system time or logical counter as timestamp
- Commonly used concurrency protocol

11.3 Transaction Isolation Levels

- Phenomena at Different Isolation Levels
 - Dirty Read: a transaction reads a data that has not yet been committed
 - Non Repeatable Read: occurs when a transaction reads same row twice and gets a different value each time
 - Phantom Read: occurs when two same queries are executed but the rows retrieved by the two are different
- SQL Standard for Isolation Levels
 - Read Uncommitted: lowest isolation level where a transaction may read not yet committed changes made by other transaction thereby allowing dirty reads (transactions are not isolated from each other)
 - Read Committed: Guarantees that any data read is committed at the moment it is read not allowing a dirty read (holds a read or write lock on current row preventing other transactions from reading, updating, or deleting)
 - Repeatable Read: Holds read locks on all rows it references and writes locks on all rows it inserts, updates, or deletes (since other transactions can't read, update, or delete these rows it avoids non-repeatable read)
 - Serializable: Highest isolation level that is guaranteed to have serializable execution (concurrently executing transactions appears to be serially executing)

11.4 Database Recovery

- Process of restoring a database to the correct state in event of failure
- **Rolling Forward** applies redo records to the corresponding data blocks
- **Rolling Back** applies rollback segments to the data
- Log-based Recovery

- Logs are the sequence of records that maintain records of actions performed by transaction
- Log of each transaction is maintained in some stable storage (can be recovered from there to recover database)
- Log contains all information about transaction being executed, values that have been modified, and transaction state
- Information stored in order of execution
- Deferred Database Modification
 - All the logs for the transaction are created and stored into stable storage system
 - Database will be updated with steps written to log
- Immediate Database Modification
 - After creating each log record, the database is modified for each step of the log entry immediately
 - Write ahead logging
- Recovery with Concurrent Transactions
 - Logs are interleaved
 - Check pointing
 - * Mechanism where all the previous logs are removed from the logging area and stored permanently in a storage system
 - * Declares a point before which the database management system was in consistent state and all the transactions were committed
 - * Point of synchronization between database and transaction log file
 - * Checkpoint record contains identifiers of all transactions that are active at time of checkpoint
- Recovery Process
 - Read log files from end to start checkpoint (can reverse transaction)
 - Maintains undo log and redo logs (for undo it uses log record to set data item specified in log record to old value and for redo it uses log record to set data item to new value)